# Deployment Guide

ECITY — Mobile Shop Management Web App

Version 1.2 · 2026

Buying the server, deploying, backups and the runbook

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

# 1. What You Are Buying, and From Whom

You need one thing: **a small Linux server on the internet**, rented by the month. That is it. Everything else — the app, the database, the web server — runs inside that one machine as Docker containers.

That kind of rented machine is called a **VPS** (Virtual Private Server) or a "cloud instance". Several companies sell them, and they are direct competitors — you buy from **one** of them, not through AWS:

| Provider | Where you buy it | Nearest region to India | 2 vCPU / 4 GB per month |
|---|---|---|---|
| **Hetzner** (recommended) | `hetzner.com/cloud` | Singapore | **≈ ₹400–600** |
| DigitalOcean | `digitalocean.com` | Bangalore, India | ≈ ₹1,060 + 18% GST |
| AWS Lightsail | `aws.amazon.com/lightsail` | Mumbai, India | ≈ ₹1,050 + 18% GST |
| Linode / Akamai | `linode.com` | Mumbai, India | ≈ ₹1,060 + 18% GST |

**Hetzner is a separate company from AWS.** You create an account on their own website and pay them directly. They bill from the EU and do not add 18% GST for Indian customers, which is part of why they land so much cheaper.

Note the two Hetzner products: **Hetzner Cloud** (`console.hetzner.cloud`) is what you want — servers by the hour, resize any time. Hetzner "Robot" sells physical dedicated machines; ignore it.

## 1.1 The one trade-off

Hetzner has no India region. Their closest is **Singapore**, about **60–90 ms** away from an Indian shop. DigitalOcean Bangalore is about **20 ms**. On a billing screen 60–90 ms is not noticeable — it is well under the 300 ms target in the PRD. If it ever bothers the staff, moving to DigitalOcean Bangalore is the same setup on a different provider, for roughly ₹800 more a month.

**Recommendation: start on Hetzner Singapore.** Switch only if latency turns out to be a real complaint.

## 1.2 Total monthly cost

| Item | ₹ / month |
|---|---|
| Hetzner CX22 server (2 vCPU, 4 GB RAM, 40 GB SSD) | ~₹400–600 |
| Hetzner automated snapshots (+20%) | ~₹90 |
| Cloudflare R2 — file storage + off-site backups (free tier, 10 GB) | ₹0 |
| Resend — transactional email (free tier, 3,000/month) | ₹0 |
| Sentry — error tracking (free tier) | ₹0 |
| Domain name (`.in`, ~₹800/year) | ~₹70 |
| **Total** | **≈ ₹550–750** |

Confirm the Hetzner Singapore price on their pricing page before you commit — regional prices differ from the German list price, and there may be a small one-time setup fee.

## 2. Day One: The Shopping List

Do these four things before touching any code. Budget about an hour.

**2.1 Create a Hetzner Cloud account** — `hetzner.com` → Cloud → Sign Up. You will need a credit/debit card or PayPal. New accounts are sometimes asked for an identity document (passport or ID card) before the first server can be created; this is normal and usually clears within a few hours. Start this first so the wait does not block you.

**2.2 Buy a domain name.** Any registrar works — Namecheap, Cloudflare Registrar, or an Indian registrar like BigRock. A `.in` domain runs about ₹800/year. You will point it at the server in §5.

**2.3 Create a Cloudflare account** (free) — `cloudflare.com`. You need it for two things: free DNS management, and **R2** object storage for backups and uploaded bill photos. R2's free tier is 10 GB of storage with zero egress fees, which will cover this shop for a long time.

**2.4 Create a GitHub account and a private repository** for the code, if you have not already. Deployments will run from here.

## 3. Creating the Server

### 3.1 Make an SSH key first

An SSH key is how you log in to the server — safer than a password, and you never type it. On your Mac, in Terminal:

```
ssh-keygen -t ed25519 -C "ecity-server"
# press Enter to accept the default path
# set a passphrase when asked, and remember it

cat ~/.ssh/id_ed25519.pub     # this prints the PUBLIC key - copy it
```

The `.pub` file is the **public** key and is safe to paste anywhere. The file without `.pub` is your **private** key — it never leaves your Mac and is never shared, committed or emailed.

### 3.2 Create the server

In the Hetzner Cloud console → **New Project** (call it `ecity`) → **Add Server**:

| Setting | Choose |
|---|---|
| Location | **Singapore** |
| Image | **Ubuntu 24.04 LTS** |
| Type | **Shared vCPU** → **CX22** (2 vCPU, 4 GB RAM, 40 GB) |
| Networking | IPv4 + IPv6 (both on) |
| SSH keys | **Add the public key you just copied** |
| Volumes / Placement | skip |
| Backups | **Enable** (+20% — do not skip this) |
| Firewall | create one, see below |
| Name | `ecity-prod` |

**Firewall rules** (inbound; everything else denied):

| Port | Protocol | Source | Why |
|---|---|---|---|
| 22 | TCP | your IP, or Any | SSH login |
| 80 | TCP | Any | HTTP (redirects to HTTPS) |
| 443 | TCP | Any | HTTPS |

Postgres port 5432 is **never** opened to the internet. The app reaches the database over Docker's internal network.

Click Create. After about 30 seconds you get an IP address like `5.223.x.x`. Write it down.

### 3.3 First login and hardening

```
ssh root@YOUR_SERVER_IP

# 1. create a normal user for day-to-day work
adduser ecity
usermod -aG sudo ecity
rsync --archive --chown=ecity:ecity ~/.ssh /home/ecity

# 2. updates and basic protection
apt update && apt upgrade -y
apt install -y ufw fail2ban unattended-upgrades
dpkg-reconfigure --priority=low unattended-upgrades

# 3. host firewall (second layer behind Hetzner's)
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw --force enable

# 4. turn off password logins entirely
sed -i 's/^#*PermitRootLogin.*/PermitRootLogin prohibit-password/' \
    /etc/ssh/sshd_config
sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication no/' \
    /etc/ssh/sshd_config
systemctl restart ssh
```

Now open a **second** terminal and confirm `ssh ecity@YOUR_SERVER_IP` works before closing the first one. If you lock yourself out, Hetzner's web console gets you back in — but check first anyway.

### 3.4 Install Docker

```
curl -fsSL https://get.docker.com | sh
usermod -aG docker ecity
# log out and back in as ecity, then verify:
docker run --rm hello-world
```

## 4. The Application Stack

Everything lives in `/home/ecity/app` on the server. Four containers:

```
  Internet :443
      |
   [caddy]      TLS certificates, automatic and free
      |
   [app]        Next.js (long-running Node process, port 3000)
      |
   [db]         PostgreSQL 16          [worker]  pg-boss jobs
```

Because the app is a **long-running process**, not serverless functions, there is no cold start and no database connection-pool problem. One small pool of ~10 connections is opened once and reused.

### 4.1 `docker-compose.yml`

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: ecity
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ecity
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecity"]
      interval: 10s
      retries: 5

  app:
    image: ghcr.io/YOURNAME/ecity:latest
    restart: unless-stopped
    env_file: .env
    depends_on:
      db: { condition: service_healthy }
    expose: ["3000"]

  worker:
    image: ghcr.io/YOURNAME/ecity:latest
    command: ["node", "dist/jobs/worker.js"]
    restart: unless-stopped
    env_file: .env
    depends_on:
      db: { condition: service_healthy }

  caddy:
    image: caddy:2-alpine
    restart: unless-stopped
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: [app]

volumes:
  db_data:
  caddy_data:
  caddy_config:
```

### 4.2 `Caddyfile`

Two lines gets you HTTPS with a free certificate that renews itself:

```
app.yourshop.in {
    encode gzip
    reverse_proxy app:3000
}
```

### 4.3 `.env` on the server

Create it with `nano .env`, then `chmod 600 .env`. **This file is never committed to git.**

```
POSTGRES_PASSWORD=<long random string>
DATABASE_URL=postgres://ecity:<same password>@db:5432/ecity
AUTH_SECRET=<openssl rand -base64 32>
AUTH_URL=https://app.yourshop.in
R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET=ecity-uploads
```

```
RESEND_API_KEY=...
SENTRY_DSN=...
NODE_ENV=production
```

Generate secrets with `openssl rand -base64 32`. Never reuse the same value twice.

### 4.4 `Dockerfile` (in the repo)

Next.js needs `output: 'standalone'` in `next.config.ts` for this to work — set that first.

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM node:22-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:22-alpine AS run
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./.next/static
COPY --from=build /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

## 5. Domain and DNS

1. In Cloudflare, add your domain and change the nameservers at your registrar to the two Cloudflare gives you. Propagation takes minutes to a few hours.
2. Add one DNS record:

| Type | Name | Value | Proxy |
|---|---|---|---|
| A | `app` | your server IP | **DNS only (grey cloud)** |

Keep the proxy **off** at first so Caddy can obtain its certificate. You can turn Cloudflare's orange-cloud proxy on afterwards if you want, but it is not needed.

3. Once DNS resolves, `docker compose up -d` and Caddy fetches a Let's Encrypt certificate automatically. `https://app.yourshop.in` is live.

## 6. Deploying

Build the image in GitHub Actions, push it to GitHub's container registry, then have the server pull it. The server never compiles anything — it only downloads and restarts, so a deploy takes seconds and a bad build never reaches production.

### 6.1 `.github/workflows/deploy.yml`

```
name: deploy
on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run typecheck && npm run lint && npm test
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: ghcr.io/YOURNAME/ecity:latest
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SSH_HOST }}
          username: ecity
          key: ${{ secrets.SSH_KEY }}
          script: |
            cd /home/ecity/app
            docker compose pull
            docker compose run --rm app npm run db:migrate
            docker compose up -d
            docker image prune -f
```

Add `SSH_HOST` (the server IP) and `SSH_KEY` (your **private** key) as GitHub repository secrets under Settings → Secrets and variables → Actions.

### 6.2 Two rules about migrations

1. **Always take a backup before a migration that drops or renames anything.** `./backup.sh` first, every time.
2. **Test every migration on staging before production.** Staging can be a second, smaller Hetzner server (CX22 ≈ ₹400) or just a second Docker stack on the same box using a different database name and port. Cheap either way, and it is what stops a bad migration reaching real sales data.

---

## 7. Backups — The Part You Cannot Skip

You are running your own database now, so backups are entirely your responsibility. This is the only real thing you gave up by not paying for managed hosting. Three layers:

| Layer | What it protects against | Frequency |
|---|---|---|
| Hetzner snapshots | Server dies, disk corrupts | Daily, automatic |
| `pg_dump` to Cloudflare R2 | Bad migration, wrong DELETE, provider outage | Nightly, off-site |
| Restore drill | Backups that were never actually valid | **Monthly, by hand** |

### 7.1 backup.sh

```
#!/usr/bin/env bash
set -euo pipefail
cd /home/ecity/app
STAMP=$(date +%F-%H%M)
OUT=/var/backups/ecity
mkdir -p "$OUT"

# -Z 0 = no Postgres compression, so restic can deduplicate
# between snapshots. Without it every nightly backup stores a
# full fresh copy and the R2 free tier fills up fast.
docker compose exec -T db \
  pg_dump -U ecity -Fc -Z 0 ecity > "$OUT/db-$STAMP.dump"

export RESTIC_REPOSITORY="s3:https://$R2_ACCOUNT_ID.r2.cloudflarestorage.com/ecity-backups"
export AWS_ACCESS_KEY_ID="$R2_ACCESS_KEY_ID"
export AWS_SECRET_ACCESS_KEY="$R2_SECRET_ACCESS_KEY"
export RESTIC_PASSWORD="$RESTIC_PASSWORD"

restic backup "$OUT"
restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 6 --prune
find "$OUT" -name 'db-*.dump' -mtime +3 -delete
```

With `-Z 0` and the retention below, seventeen retained snapshots deduplicate down to roughly the size of one or two — which is what keeps the whole backup history inside Cloudflare R2's free 10 GB.

Install with `apt install -y restic`, initialise once with `restic init`, then `chmod +x backup.sh` and schedule it — `crontab -e`:

```
# every four hours, so a crash costs at most one quarter of a trading day
30 */4 * * * /home/ecity/app/backup.sh >> /var/log/ecity-backup.log 2>&1
```

A nightly-only schedule means a disk failure at 8 PM loses **everything entered that day** across every branch. Four-hourly costs nothing extra and cuts the worst case to about four hours. For true point-in-time recovery — restoring to any moment, including just before a bad migration — add WAL archiving with `pgBackRest` or `wal-g` shipping to the same R2 bucket. That is roughly half a day of setup in M13 and brings the worst case down to about five minutes.

### 7.2 The monthly restore drill

Put a recurring reminder in your calendar. It takes ten minutes.

```
# 1. pull the latest backup down
restic restore latest --target /tmp/restore

# 2. load it into a scratch database
docker compose exec -T db createdb -U ecity ecity_drill
docker compose exec -T db pg_restore -U ecity -d ecity_drill \
  /tmp/restore/var/backups/ecity/db-<stamp>.dump

# 3. confirm it is real data, not an empty shell
docker compose exec -T db psql -U ecity -d ecity_drill \
  -c "select count(*) from sale; select count(*) from device_unit;"

# 4. clean up and note how long the whole thing took
docker compose exec -T db dropdb -U ecity ecity_drill
```

If the counts look right, your backups work. If you have never done this, **you do not have backups** — you have files you hope are backups.

## 8. Monitoring

| What | Tool | Cost |
|---|---|---|
| Application errors | Sentry (free Developer tier) | ₹0 |
| Site down alert | UptimeRobot, 5-minute checks to email/SMS | ₹0 |
| Disk full | `df -h` in a weekly cron that emails if above 80% | ₹0 |
| Backup failed | Have `backup.sh` email or Sentry-alert on non-zero exit | ₹0 |

Disk filling up is the single most common way a small VPS falls over. Old Docker images are usually the cause — `docker image prune -f` runs on every deploy in the workflow above, which handles it.

## 9. Runbook

Common operations, for when you need them and cannot remember.

```
# where everything lives
cd /home/ecity/app

# see what is running / logs
docker compose ps
docker compose logs -f app
docker compose logs --tail=200 worker

# restart one service
docker compose restart app

# deploy by hand (normally GitHub Actions does this)
docker compose pull && docker compose up -d

# roll back to the previous image
docker compose pull ghcr.io/YOURNAME/ecity:<previous-sha>
# edit docker-compose.yml to that tag, then:
docker compose up -d

# open a database shell
docker compose exec db psql -U ecity ecity

# take a backup right now
./backup.sh

# disk and memory
df -h && free -m && docker system df
```

**Resizing the server** (when 10 branches actually arrive): Hetzner console → Server → Rescale → CX32 (4 vCPU / 8 GB). It reboots once, takes about a minute, and nothing else changes. You can scale up freely; scaling disk down is not possible, so do not over-provision the disk early.

## 10. Go-Live Checklist

- [ ] Hetzner account verified, CX22 in Singapore running, snapshots enabled
- [ ] Hetzner firewall: only 22, 80, 443 inbound. Port 5432 closed
- [ ] SSH key login working; password login disabled; `ufw` enabled

- [ ] Domain pointing at the server; HTTPS live with a valid certificate
- [ ] `.env` present, `chmod 600`, and **not** in git
- [ ] All secrets freshly generated; none reused from development
- [ ] GitHub Actions deploys on push to `main`; tests gate the deploy
- [ ] Staging environment exists and migrations are tested there first
- [ ] `backup.sh` scheduled nightly and confirmed writing to R2
- [ ] **A restore drill completed successfully and timed at least once**
- [ ] Sentry receiving errors; UptimeRobot alerting to a phone
- [ ] Opening balances loaded (PRD FR-34)
- [ ] Two-week parallel run with paper records agreed with the shop

## 11. Things That Will Bite You

| Mistake | What happens | Prevention |
|---|---|---|
| Never testing a restore | You discover the backups were empty on the day you need them | The monthly drill in §7.2 |
| Exposing Postgres port 5432 | The database is found and attacked within hours | Never publish port 5432; Docker's internal network only |
| Secrets committed to git | Credentials leak permanently, even after deletion | `.env` in `.gitignore`; GitHub secrets for CI |
| Running migrations straight on production | One bad migration, no way back | Staging first, backup immediately before |
| Disk quietly filling up | Postgres stops accepting writes; the shop stops billing | `docker image prune` on deploy; weekly disk check |
| Editing files on the server by hand | The next deploy silently overwrites your fix | All changes go through git and CI, always |
| Only one person holds the SSH key | Nobody can reach the server if that laptop dies | Second key stored securely offline, or Hetzner console access |